

Report:

Identifying_Key_Entities_Recipe_Data

Include your visualizations, analysis, results, insights, and outcomes. Explain your methodology and approach to the tasks. Add your conclusions to the sections.

1. Import necessary libraries

- All the necessary libraries were imported

```
: # Import necessary Libraries
import json # For handling JSON data
import pandas as pd # For data manipulation and analysis
import re # For regular expressions (useful for text preprocessing)
import matplotlib.pyplot as plt # For visualisation
import seaborn as sns # For advanced data visualisation
import sklearn_crfsuite # CRF (Conditional Random Fields) implementation for sequence modeling
import numpy as np # For numerical computations
# Saving and loading machine Learning models
import joblib
import random
import spacy
from IPython.display import display, Markdown # For displaying well-formatted output

from fractions import Fraction # For handling fractional values in numerical data
# Importing tools for feature engineering and model training
from collections import Counter # For counting occurrences of elements in a list
from sklearn.model_selection import train_test_split # For splitting dataset into train and test sets
from sklearn_crfsuite import metrics # For evaluating CRF models
from sklearn_crfsuite.metrics import flat_classification_report
from sklearn.utils.class_weight import compute_class_weight
from collections import Counter
from sklearn.metrics import confusion_matrix
```

2. Data Ingestion and Preparation

2.1. Read recipe data from Dataframe and prepare the data for analysis:

The data is in the form of a json file. The dataframe contained 285 rows and 2 columns.

	input	pos
0	6 Karela Bitter Gourd Pavakkai Salt 1 Onion 3 tablespoon Gram flour besan 2 teaspoons Turmeric powder Haldi Red Chilli Cumin seeds Jeera Coriander Powder Dhania Amchur Dry Mango Sunflower Oil	quantity ingredient ingredient ingredient ingredient ingredient quantity ingredient quantity unit ingredient ingredient ingredient quantity unit ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient
1	2-1/2 cups rice cooked 3 tomatoes teaspoons BC Belle Bhat powder 1 teaspoon chickpea lentils 1/2 cumin seeds white urad dal mustard green chilli dry red 2 cashew or peanuts 1-1/2 tablespoon oil asafoetida	quantity unit ingredient ingredient quantity ingredient unit ingredient ingredient ingredient ingredient ingredient quantity unit ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient
2	1-1/2 cups Rice Vermicelli Noodles Thin 1 Onion sliced 1/2 cup Carrots Gajjar chopped 1/3 Green peas Matar 2 Chillies 1/4 teaspoon Asafoetida hing Mustard seeds White Urad Dal Split Ghee sprig Curry leaves Salt Lemon juice	quantity unit ingredient ingredient ingredient ingredient quantity ingredient ingredient quantity unit ingredient ingredient ingredient quantity unit ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient
3	500 grams Chicken 2 Onion chopped 1 Tomato 4 Green Chillies slit inch Ginger finely 6 cloves Garlic 1/2 teaspoon Turmeric powder Haldi Garam masala tablespoon Sesame Gingelly Oil 1/4 Methi Seeds Fenugreek Coriander Dhania Dry Red Fennel seeds Saunf cups Sorrel Leaves Gongura picked and	quantity unit ingredient quantity ingredient ingredient quantity ingredient quantity ingredient ingredient ingredient ingredient unit ingredient ingredient ingredient quantity ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient
4	1 tablespoon chana dal white urad 2 red chillies coriander seeds 3 inches ginger onion tomato Teaspoon mustard asafoetida sprig curry	quantity unit ingredient ingredient ingredient ingredient quantity ingredient ingredient ingredient ingredient ingredient quantity unit ingredient ingredient ingredient unit ingredient ingredient unit ingredient

2.2 Recipe Data Manipulation

2.2.1. Input_tokens and pos_tokens columns were created by splitting the input and pos from the dataframe

	input	pos	input_tokens	pos_tokens
0	6 Karela Bitter Gourd Pavakkai Salt 1 Onion 3 tablespoon Gram flour besan 2 teaspoons Turmeric powder Haldi Red Chilli Cumin seeds Jeera Coriander Powder Dhania Amchur Dry Mango Sunflower Oil	quantity ingredient ingredient ingredient ingredient ingredient quantity ingredient quantity unit ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient	[6, Karela, Bitter, Gourd, Pavakkai, Salt, 1, Onion, 3, tablespoon, Gram, flour, besan, 2, teaspoons, Turmeric, powder, Haldi, Red, Chilli, Cumin, seeds, Jeera, Coriander, Powder, Dhania, Amchur, Dry, Mango, Sunflower, Oil]	[quantity, ingredient, ingredient, ingredient, ingredient, ingredient, quantity, ingredient, quantity, unit, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient]
1	2-1/2 cups rice cooked 3 tomatoes teaspoons BC Belle Bhat powder 1 teaspoon chickpea lentils 1/2 cumin seeds white urad dal mustard green chilli dry red 2 cashew or peanuts 1-1/2 tablespoon oil asafoetida	quantity unit ingredient ingredient quantity ingredient unit ingredient ingredient ingredient quantity ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient	[2-1/2, cups, rice, cooked, 3, tomatoes, teaspoons, BC, Belle, Bhat, powder, 1, teaspoon, chickpea, lentils, 1/2, cumin, seeds, white, urad, dal, mustard, green, chilli, dry, red, 2, cashew, or, peanuts, 1-1/2, tablespoon, oil, asafoetida]	[quantity, unit, ingredient, ingredient, quantity, ingredient, unit, ingredient, ingredient, ingredient, ingredient, quantity, unit, ingredient, ingredient, quantity, ingredient, ingredient, ingredient, ingredient, ingredient, quantity, ingredient, ingredient, quantity, unit, ingredient, ingredient]
2	1-1/2 cups Rice Vermicelli Noodles Thin 1 Onion sliced 1/2 cup Carrots Gajjar chopped 1/3 Green peas Matar 2 Chillies 1/4 teaspoon Asafoetida hing Mustard seeds White Urad Dal Split Ghee sprig Curry leaves Salt Lemon juice	quantity unit ingredient ingredient ingredient quantity unit ingredient ingredient ingredient quantity ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient	[1-1/2, cups, Rice, Vermicelli, Noodles, Thin, 1, Onion, sliced, 1/2, cup, Carrots, Gajjar, chopped, 1/3, Green, peas, Matar, 2, Chillies, 1/4, teaspoon, Asafoetida, hing, Mustard, seeds, White, Urad, Dal, Split, Ghee, sprig, Curry, leaves, Salt, Lemon, juice]	[quantity, unit, ingredient, ingredient, ingredient, ingredient, quantity, ingredient, ingredient, quantity, unit, ingredient, ingredient, ingredient, quantity, ingredient, quantity, unit, ingredient, ingredient, ingredient, ingredient, unit, ingredient, ingredient, ingredient, ingredient]

2.2.2. Provide the length for input_tokens and pos_tokens and validate their Length

Out of the total dataset, five rows showed unequal lengths between input tokens and POS tokens. This mismatch is primarily due to tokenisation issues involving fractional quantities, special characters, and multi-word ingredients. Such inconsistencies are common in real-world NLP datasets and must be handled before sequence modelling.

	input_length	pos_length
0	31	31
1	34	34
2	37	37
3	46	46
4	21	21

```
length_match
True      280
False       5
Name: count, dtype: int64
```

2.2.3. Define a unique_labels function and validate the labels in pos_tokens and Provide the insights seen in the recipe data after validation

After validating the recipe dataset, the following insights were observed:

i. The dataset contains three unique POS labels: ingredient, quantity, and unit.

```
{'ingredient', 'quantity', 'unit'}
```

ii. The label distribution is imbalanced, with ingredient being the most frequent label (ingredient: 7543, quantity: 1414, unit: 1190).

```
Counter({'ingredient': 7543, 'quantity': 1414, 'unit': 1190})
```

iii. Five rows in the dataset have unequal lengths between input_tokens and pos_tokens, indicating token-label misalignment. These mismatches are caused by fractional quantities, special characters, and multi-word ingredients, which require cleaning and formatting before sequence modelling.

iv. Indexes that require cleaning and formatting: The following row indexes require preprocessing due to unequal token lengths: 17, 27, 79, 164, 207

```
| Indexes needing cleaning: [17, 27, 79, 164, 207]
```

2.2.4. Drop the rows that have invalid data provided in previous cell

Rows with unequal token lengths were dropped. The final dataframe contained 280 rows.

```
length_match
True      280
Name: count, dtype: int64
```

2.2.5. The input_length & pos_length in dataframe were updated

	input_length	pos_length
0	31	31
1	34	34
2	37	37
3	46	46
4	21	21

2.2.6. Validate the input_length and pos_length by checking unequal rows

```
# validate the input length and pos length as input_length and pos_length
(df["input_length"] == df["pos_length"]).value_counts()
```

```
[46]:
```

```
True      280
Name: count, dtype: int64
```

3. Train Validation Split (70 train - 30 val)

3.1. Perform train and validation split ratio

The dataset was split into train_df and val_df into 70:30 ratio

The dataset was extracted into train_df and val_df into X_train, X_val, y_train and y_val and

displayed their length. The training dataset contains three unique labels: ingredient, quantity, and unit, confirming consistency with the overall dataset.

```
train_df shape: (196, 7)
val_df shape: (84, 7)
```

4. Exploratory Recipe Data Analysis on Training Dataset

The following steps were performed:

- Flattened the lists for input_tokens & pos_tokens
- Extracted and validate the tokens after using the flattening technique
- Categorised tokens into labels (unit, ingredient, quantity)
- Top 10 Most Frequent Items were identified
- Then performed EDA analysis

The most frequent ingredients in the training dataset are powder, salt, seeds, Green, and

chopped, indicating that spices, seasonings, and commonly used vegetables dominate the recipes.

After normalisation, the most common units in the training dataset are teaspoon , cup , and tablespoon, with other units like gram , inch, sprig , and clove used less frequently, reflecting a mix of standard volume, weight, and count-based measurements.

Training Dataset

Total input tokens: 7114

Total POS tokens: 7114

True

input tokens:

['250', 'grams', 'Okra', 'Oil', '1', 'Onion', 'finely', 'chopped', 'Tomato', 'Grated']

POS tokens:

['quantity', 'unit', 'ingredient', 'ingredient', 'quantity', 'ingredient', 'ingredient', 'ingredient', 'ingredient', 'ingredient']

ingredients count: 5323

units count: 811

quantities count: 980

First 10 ingredients: ['Okra', 'Oil', 'Onion', 'finely', 'chopped', 'Tomato', 'Grated', 'Ginger', 'Garlic', 'Finely']

First 10 units: ['grams', 'teaspoon', 'Teaspoon', 'cup', 'grams', 'tablespoon', 'teaspoon', 'grams', 'teaspoon', 'sprig']

First 10 quantities: ['250', '1', '2', '1/2', '1/4', '200', '2', '1', '1/2', '500']

Top 10 units after dataset normalization: [('teaspoon', 212), ('cup', 186), ('tablespoon', 164), ('gram', 66), ('inch', 53), ('sprig', 44), ('clove', 40), ('pinch', 14), ('spoon', 4), ('small', 4)]

Most frequent_Top10 ingredients in Training dataset:

powder: 129

Salt: 102

seeds: 89

Green: 85

chopped: 84

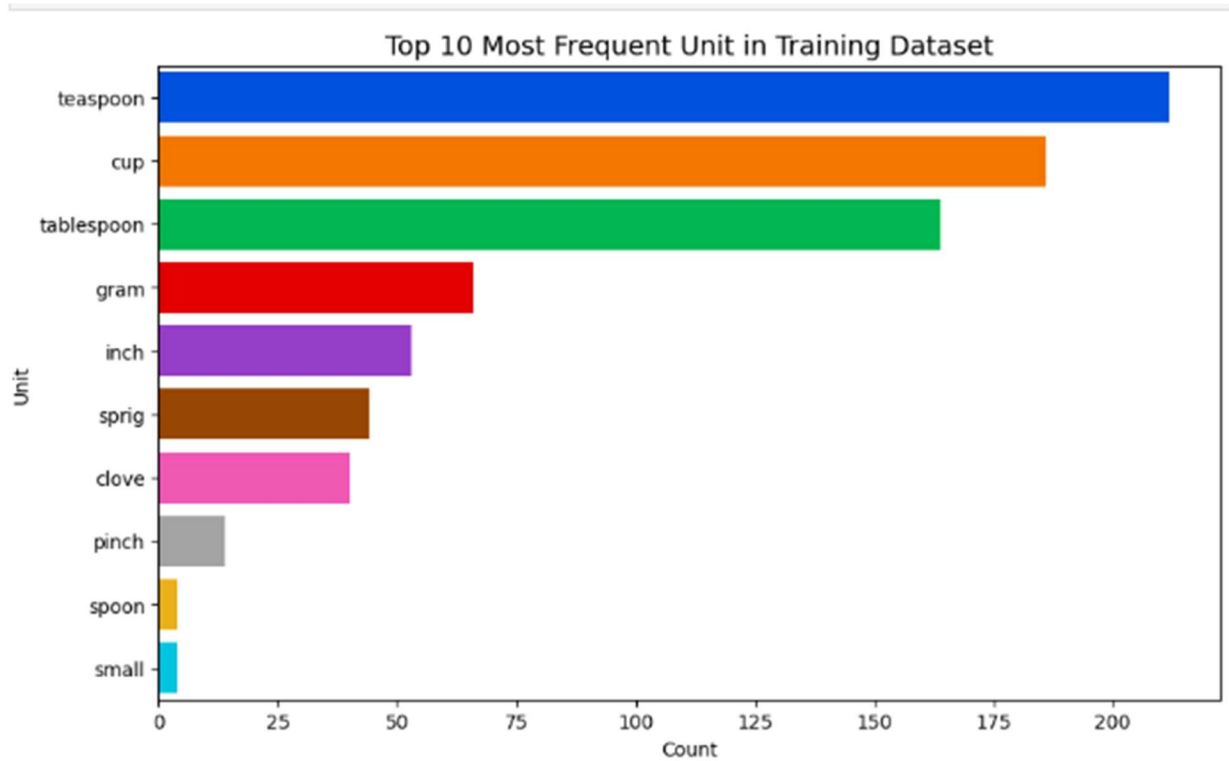
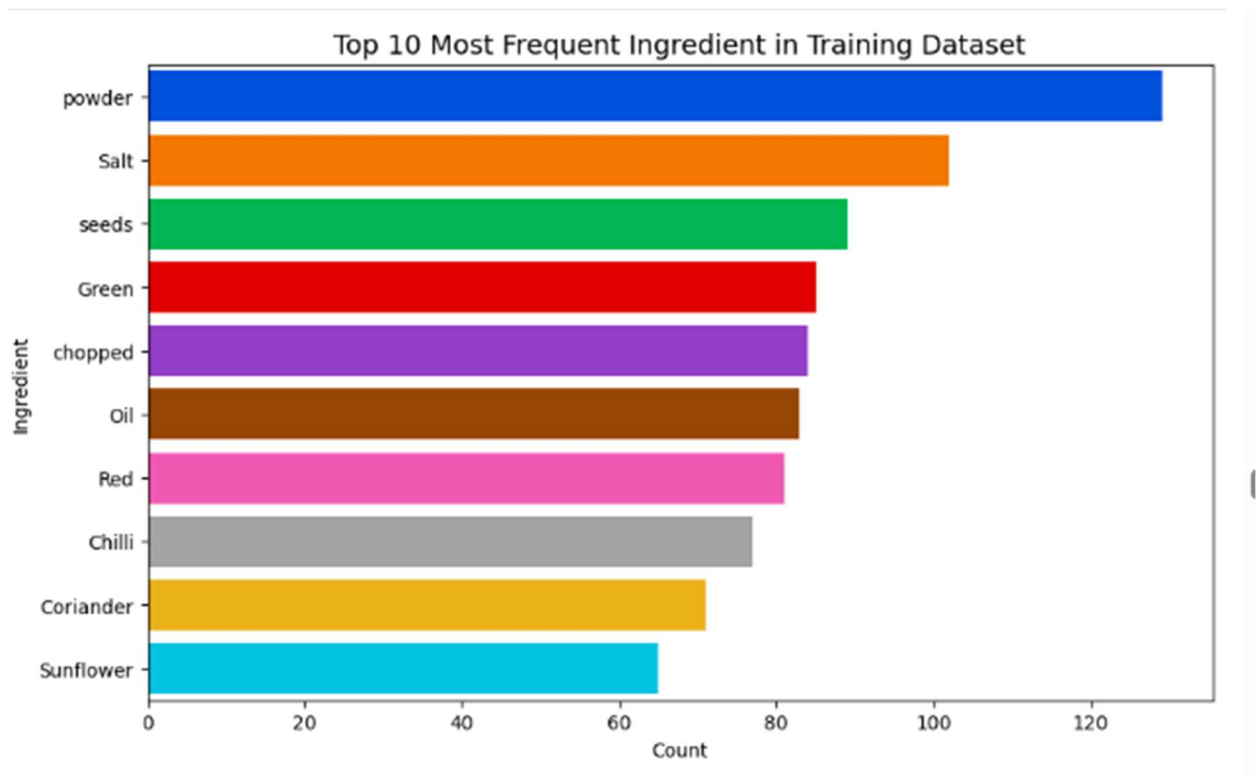
Oil: 83

Red: 81

Chilli: 77

Coriander: 71

Sunflower: 65



5. Exploratory Recipe Data Analysis on Validation Dataset (Optional)

Validation Dataset

Total input tokens: 2876

Total POS tokens: 2876

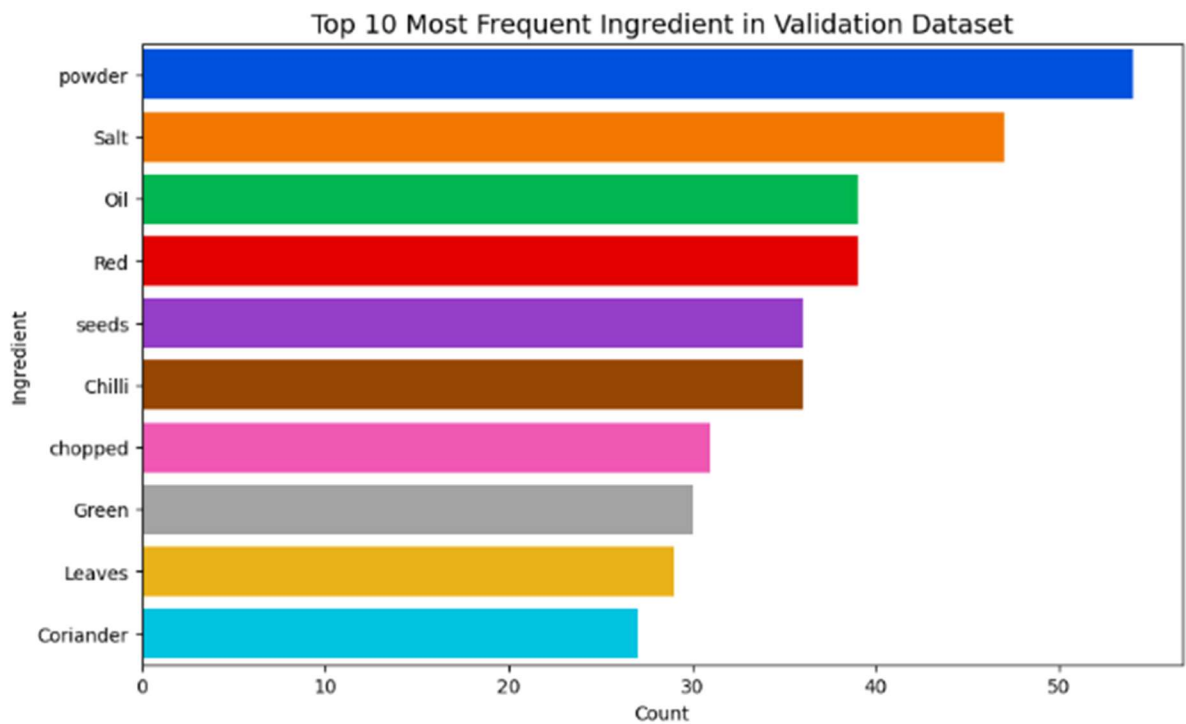
True

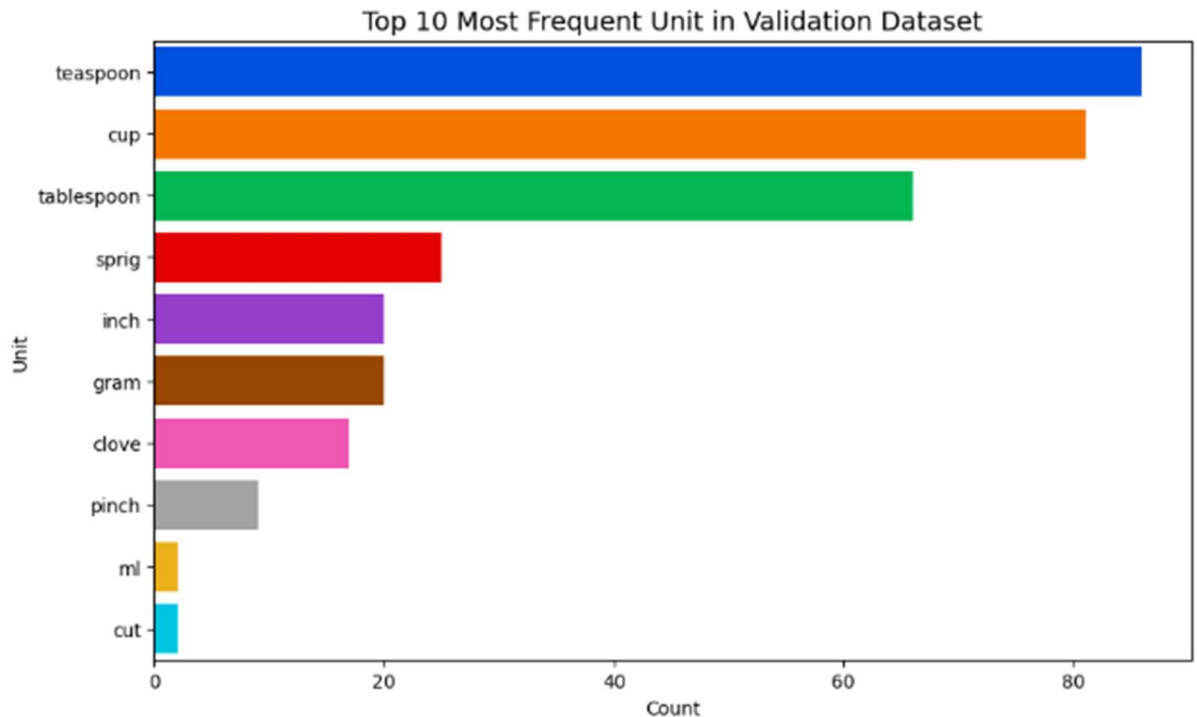
input tokens:

['1', 'cup', 'Ada', '2', 'liter', 'Milk', '3/4', 'Sugar', 'tablespoon', 'Ghee']

POS tokens:

['quantity', 'unit', 'ingredient', 'quantity', 'unit', 'ingredient', 'quantity', 'ingredient', 'unit', 'ingredient']





6. Feature Extraction For CRF Model

Feature Extraction for CRF Model was done using following steps

6.1. Define a feature functions to take each token from recipe

6.1.1. Define keywords for unit and quantity and create a quantity pattern to work on fractions, numbers and decimals

Number of unit keywords: 29

```
{'cut', 'raw', 'whole', 'scoops', 'handful', 'tablespoons', 'pinch', 'spoon', 'drops',
'tablespoon', 'finely', 'long', 'cup', 'chopped', 'stalks', 'clove', 'teaspoon', 'inc
h', 'wedges', 'fistful', 'liter', 'gram', 'thick', 'ml', 'small', 'sprig', 'for', 'or',
'thin'}
```

Number of quantity keywords: 59

```
{'1-1/2', '3', '5', '1/3', '2.5', 'per', '2-1', '1-3/4', '100', '125', '18', '2-3',
'7', '80', 'a', '250', '150', 'two', '12', '1', '1/2', '15', 'finely', 'few', 'to', '5
0', '2-1/2', '20', '200', '2', '6', '10', '4-5', '75', '600', '1-2', '160', '1.5', '3/
4', '500', '300', '3-4', '4', '1/8', '1-1/4', 'one', '30', 'taste', '350', '225', '5-
6', '8', '24', '400', '175', 'for', 'little', '1/4', 'as'}
```

6.1.2. Defined feature functions for CRF

6.2. Preparation of Recipe level features

6.2.1. Define function to work on all the recipes and call word2features for each Recipe

6.3. Convert X_train, X_val, y_train and y_val into train and validation feature sets and labels

6.3.1. Converted recipe into feature functions by using X_train and X_val

6.3.2. Convert labels of y_train and y_val into list

```
First training labels: ['quantity', 'unit', 'ingredient', 'ingredient', 'quantity', 'ingredient', 'ingredient', 'ingredient', 'ingredient', 'ingredient', 'unit', 'ingredient', 'quantity', 'ingredient', 'ingredient', 'quantity', 'ingredient', 'ingredient', 'quantity', 'unit', 'ingredient', 'unit', 'ingredient', 'ingredient', 'ingredient', 'ingredient', 'ingredient', 'ingredient', 'ingredient', 'ingredient', 'ingredient']
First validation labels: ['quantity', 'unit', 'ingredient', 'quantity', 'unit', 'ingredient', 'quantity', 'ingredient', 'unit', 'ingredient', 'quantity', 'unit', 'ingredient', 'ingredient', 'ingredient']
```

6.3.3. Print the length of val and train features and labels

Training: 196 sentences → features and labels match Validation: 84 sentences → features and labels match This means your CRF dataset is ready for model training.

```
Number of training feature sentences: 196
Number of training label sentences: 196
```

```
Number of validation feature sentences: 84
Number of validation label sentences: 84
```

6.4. Applying weights to feature sets

6.4.1. Flattened the labels of y_train

```
Flattened labels_first_10: ['quantity', 'unit', 'ingredient', 'ingredient', 'quantity', 'ingredient', 'ingredient', 'ingredient', 'ingredient', 'ingredient']
```

6.4.2. Count the labels present in training target dataset

```
Label_counts: Counter({'ingredient': 5323, 'quantity': 980, 'unit': 811})
Total_samples: 7114
```

6.4.3. Computed the weight_dict by using inverse frequency method for label weights

```
Class weights: {'quantity': 7.259183673469388, 'unit': 8.771886559802713, 'ingredient': 2.0046965996618447}
```

6.4.4. Extracted features along with class weights

6.4.5. Executed extract_features_with_class_weights on training and validation Datasets

```
First training weighted labels: [('quantity', 7.259183673469388), ('unit', 8.771886559802713), ('ingredient', 2.0046965996618447), ('ingredient', 2.0046965996618447), ('quantity', 7.259183673469388), ('ingredient', 2.0046965996618447), ('ingredient', 2.0046965996618447), ('ingredient', 2.0046965996618447), ('ingredient', 2.0046965996618447), ('ingredient', 2.0046965996618447), ('unit', 8.771886559802713), ('ingredient', 2.0046965996618447), ('quantity', 7.259183673469388), ('ingredient', 2.0046965996618447), ('ingredient', 2.0046965996618447), ('quantity', 7.259183673469388), ('ingredient', 2.0046965996618447), ('quantity', 7.259183673469388), ('unit', 8.771886559802713), ('ingredient', 2.0046965996618447), ('unit', 8.771886559802713), ('ingredient', 2.0046965996618447), ('ingredient', 2.0046965996618447), ('ingredient', 2.0046965996618447), ('ingredient', 2.0046965996618447), ('ingredient', 2.0046965996618447), ('ingredient', 2.0046965996618447), ('ingredient', 2.0046965996618447), ('ingredient', 2.0046965996618447), ('ingredient', 2.0046965996618447)]
First training token feature dict: {'bias': 1.0, 'token': '250', 'lemma': '250', 'pos_tag': 'NUM', 'tag': 'CD', 'dep': 'ROOT', 'shape': 'ddd', 'is_stop': False, 'is_digit': True, 'has_digit': True, 'has_alpha': False, 'hyphenated': False, 'slash_present': False, 'is_title': False, 'is_upper': False, 'is_punct': False, 'is_quantity': True, 'is_unit': False, 'is_numeric': True, 'is_fraction': False, 'is_decimal': False, 'preceding_word': '', 'following_word': 'grams', 'BOS': True, 'EOS': False, 'prev_token': '', 'prev_is_quantity': False, 'prev_is_digit': False, 'next_token': 'grams', 'next_is_unit': False, 'next_is_ingredient': True}
```

7. Model Building and Training

7.1. Initialised the CRF model and train it

```
CRF
CRF(algorithm='lbfgs', all_possible_transitions=True, c1=0.5, c2=1.0, max_iterations=100)
```

7.2. Evaluation of Training Dataset using CRF model

Training Data - Classification Report:

	precision	recall	f1-score	support
ingredient	0.9920	0.9962	0.9941	5323
quantity	0.9959	0.9857	0.9908	980
unit	0.9799	0.9642	0.9720	811
accuracy			0.9911	7114
macro avg	0.9893	0.9821	0.9856	7114
weighted avg	0.9911	0.9911	0.9911	7114

Training Data_Confusion Matrix:

[363]:

	quantity	unit	ingredient
quantity	966	0	14
unit	0	782	29
ingredient	4	16	5303

7.3. Saved the CRF model

```
# dump the model using joblib as crf_model.pkl
joblib.dump(crf, "crf_model.pkl")
print("CRF model saved")
```

CRF model saved

8. Prediction and Model Evaluation

8.1. Predict and Evaluate the CRF model on validation set

Insights:

- The small difference between training and validation confusion matrices indicates minimal overfitting.
- Class weighting and contextual features effectively addressed class imbalance.
- The CRF successfully leveraged contextual dependencies (e.g., quantity → unit → ingredient patterns).
- The CRF model demonstrates high accuracy, robust generalisation, and effective sequence learning,
- making it well-suited for ingredient parsing and recipe information extraction tasks.

Validation Data_Classification Report:

	precision	recall	f1-score	support
ingredient	0.9803	0.9943	0.9873	2107
quantity	0.9878	0.9854	0.9866	411
unit	0.9726	0.8939	0.9316	358
accuracy			0.9805	2876
macro avg	0.9803	0.9579	0.9685	2876
weighted avg	0.9805	0.9805	0.9802	2876

Validation Data_Confusion Matrix:

[371]:

	quantity	unit	ingredient
quantity	405	1	5
unit	1	320	37
ingredient	4	8	2095

9. Error Analysis on Validation Data

9.1. Investigate misclassified samples in validation dataset

9.1.1. Flattened the labels of validation data and initialize error data

9.1.2. Iterated the validation data and collect Error Information

9.1.3. Created dataframe from error_data and print overall accuracy

Overall_Validation_Accuracy: 98.05 %

[382]:

	token	prev_token	next_token	true_label	predicted_label	context	class_weight
0	few	Leaves	<EOS>	ingredient	quantity	1 tablespoon Sunflower Oil 3 Potato Aloo Ginger paste Green Chilli chopped 1-1/12 tablespoons Sesame seeds	2.004697
						Til teaspoon Red powder Cumin Jeera Coriander Powder Dhania 1/2 Garam masala 2 Sweet Chutney Date Tamarind Leaves few	
1	chillies	2	turmeric	ingredient	unit	1 cup green peas gram flour 1/2 cheese tsp ginger 2 chillies turmeric powder cumin teaspoon salt oil	2.004697
						1 cup cabbage leaves 3/4	

9.1.4. Analyse errors by label type

	label	class_weight	accuracy	total_samples	errors
0	quantity	7.259184	98.54	411	6
1	unit	8.771887	89.39	358	38
2	ingredient	2.004697	99.43	2107	12

token	prev_token	next_token	true_label	predicted_label	context	class_weight
					1 tablespoon Sunflower Oil 3 Potato Aloo Ginger paste Green Chilli chopped 1- 1/12 tablespoons Sesame seeds Til teaspoon	
0	few	Leaves	<EOS>	ingredient	quantity	2.004697
					Red powder Cumin Jeera Coriander Powder Dhania 1/2 Garam masala 2 Sweet Chutney Date Tamarind Leaves few	

9.2. Provide insights from the validation dataset

- The CRF model achieved a high overall validation accuracy of 98.05%, indicating strong generalisation on unseen recipe data.
- Ingredient labels show the highest accuracy (99.43%), demonstrating effective handling of class imbalance.
- Quantity labels are predicted accurately (98.54%), confirming that numeric patterns and contextual cues are well learned by the model.
- Unit labels have the lowest accuracy (89.39%), mainly due to semantic ambiguity with ingredient and preparation-related tokens.
- Most misclassifications occur between unit and ingredient classes, particularly for words like cloves, spoon, cut, sliced, and julienned.
- Context-dependent terms such as few, to, and +1/3 contribute to occasional quantityingredient confusion.

Internal

- Class weighting helped reduce ingredient dominance and improved recognition of minority classes.
- Overall, errors are caused more by linguistic ambiguity than by model limitations, indicating a robust CRF implementation.

10. Conclusion (Optional)

- A Conditional Random Field (CRF) model was successfully developed to identify quantity, unit, and ingredient entities from recipe text.
 - Rich token-level, contextual, and domain-specific features enabled the model to effectively capture sequential patterns present in recipes.
 - The use of class weighting helped address label imbalance and improved recognition of minority classes.
 - The model achieved high performance, with 98.05% validation accuracy, demonstrating strong generalisation on unseen data.
 - Error analysis revealed that most misclassifications occurred due to semantic ambiguity between unit and ingredient tokens rather than model deficiencies.
 - Quantity detection showed consistently strong performance due to clear numeric patterns and contextual cues.
 - Overall, the CRF model proved to be robust, reliable, and well-suited for structured information extraction from recipe datasets.
 - Future improvements could include incorporating semantic embeddings and expanding unit vocabularies to further reduce ambiguity.
- approximately linear.